

bash

#!/bin/bash

Bu bir **shebang** satırı. Script'in hangi yorumlayıcı (interpreter) ile çalıştırılacağını belirtir — burada /bin/bash. Dosya çalıştırılabilir yapıp (chmod +x) doğrudan ./user_password.sh şeklinde çağrıldığında sistem bu satırı okuyup bash'i devreye sokar.

bash

```
if [ "$(id -u)" -ne 0 ]; then
```

```
    echo "Error: You must run this script as root (use sudo)." >&2
```

```
    exit 1
```

```
fi
```

- id -u → çalıştıran kullanıcının **UID**'sini verir. root'un UID'si her zaman 0'dır.
- \$(...) → komut ikamesi (command substitution), yani id -u komutunun çıktısını bir değişken gibi kullanır.
- -ne 0 → "not equal" yani UID 0'a eşit değilse (root değilse).
- >&2 → hata mesajını standart çıktı yerine **standart hata (stderr)** akışına yönlendirir; bu, hata mesajlarının doğru kanaldan gitmesi için genel bir kuraldır.
- exit 1 → script'i "başarısız" anlamına gelen 1 kodu ile sonlandırır. useradd, chpasswd gibi komutlar root yetkisi ister, o yüzden en başta kontrol ediliyor.

bash

```
if [ -z "$1" ]; then
```

```
    read -rp "Enter the username to create: " USER_NAME
```

```
else
```

```
    USER_NAME="$1"
```

```
fi
```

- \$1 → script'e verilen **birinci parametre** (kullanıcı adı).
- -z "\$1" → string'in **boş (zero length)** olup olmadığını kontrol eder. Yani script parametresiz çağrıldıysa bu doğru olur.
- Eğer parametre verilmediyse: read -rp "." ile kullanıcıdan interaktif olarak isim ister (-r backslash'leri ham okur, -p prompt mesajı gösterir), cevabı USER_NAME değişkenine atar.
- Parametre verildiyse: doğrudan \$1'i USER_NAME'e atar.

bash

```
if [ -z "$2" ]; then
```

```
    read -rp "Enter the comment (e.g. full name) for this account: " COMMENT
```

```
else
```

shift

COMMENT="\$*"

fi

- \$2 → **ikinci parametre** (yorum/açıklama, örn. gerçek isim).
- Verilmediyse yine interaktif sorulur.
- Verildiyse: shift komutu parametre listesini bir kaydırır (yani eski \$2 artık \$1 olur, eski \$1 düşer). Böylece \$* (kalan tüm parametreler) sadece yorum kısmını içerir. Bu, "John Doe" gibi **birden fazla kelimedenden oluşan** bir yorumun tırnaksız girilse bile (./script.sh john John Doe) doğru şekilde birleştirilmesini sağlar.

bash

PASSWORD=\$(tr -dc 'A-Za-z0-9!@#\$\$%^&*()_+' </dev/urandom | head -c 12)

- /dev/urandom → çekirdeğin rastgele bayt (byte) ürettiği özel bir cihaz dosyası; şifreleme amaçlı rastgelelik kaynağı olarak kullanılır.
- tr -dc 'A-Za-z0-9!@#\$\$%^&*()_+' → gelen rastgele baytlardan **sadece** belirtilen karakter setindekileri (harf, rakam, bazı özel karakterler) bırakır, geri kalanını siler (-d delete, -c complement yani "belirtilenler dışındakileri" sil).
- head -c 12 → sonuçtan ilk **12 karakteri** keser.
- Bu üç komut boru (|) ile birbirine bağlanarak rastgele, 12 karakterlik bir şifre üretir ve PASSWORD değişkenine atanır.

bash

useradd -c "\$COMMENT" -m "\$USER_NAME"

- useradd → yeni kullanıcı hesabı oluşturan komut.
- -c "\$COMMENT" → **comment/GECOS** alanına açıklama (genelde gerçek ad) yazar (/etc/passwd dosyasında görünür).
- -m → kullanıcı için /home/<kullanıcı> dizinini otomatik oluşturur (**m**ake home directory).

bash

if [\$? -ne 0]; then

echo "Error: Could not create user account \$USER_NAME." >&2

exit 1

fi

- \$? → **bir önceki komutun çıkış kodu**. 0 başarı demektir, 0 dışı bir değer hata demektir.
- Burada useradd başarısız olduysa (örn. kullanıcı zaten varsa) hata verip script'i durdurur. Bu kontrol, "kullanıcı oluşturulmadığı halde başarılı mesajı gösterme" riskini önler.

bash

```
echo "$USER_NAME:$PASSWORD" | chpasswd
```

- chpasswd → kullanıcı:şifre formatındaki girdiyi stdin'den okuyup şifreyi ayarlayan komut.
- echo "\$USER_NAME:\$PASSWORD" bu formatı üretir, | ile chpasswd'a boru hattıyla aktarılır. Bu yöntem, passwd komutunun interaktif (iki kere şifre sorma) davranışını atlayarak script içinde otomatik şifre atamayı mümkün kılar.

```
bash
```

```
if [ $? -ne 0 ]; then
```

```
    echo "Error: Could not set password for $USER_NAME." >&2
```

```
    exit 1
```

```
fi
```

Aynı mantık — chpasswd başarısız olduysa hata verip çıkar.

```
bash
```

```
passwd -e "$USER_NAME"
```

- -e (**expire**) → kullanıcının mevcut şifresini "süresi dolmuş" olarak işaretler. Sonuç: kullanıcı ilk giriş yaptığında sistem otomatik olarak yeni bir şifre belirlemesini ister. Bu, görevde istenen "ilk girişte şifre değiştirme zorunluluğu" kısmını karşılıyor.

```
bash
```

```
echo "-----"
```

```
echo "User account created successfully!"
```

```
echo "Host    : $(hostname)"
```

```
echo "Username : $USER_NAME"
```

```
echo "Password : $PASSWORD"
```

```
echo "Note: user must change this password at first login."
```

```
echo "-----"
```

- \$(hostname) → makinenin adını çıktısına verir; bu, hangi sunucuda hesabın oluşturulduğunu göstermek için kullanılıyor.
- Geri kalan satırlar, oluşturulan kullanıcı adı, üretilen şifre ve host bilgisini ekrana düzenli (çerçevesiz) biçimde yazdırıp özetliyor — böylece admin bu bilgiyi kopyalayıp ilgili kişiye iletebilir.

Genel akış özet: **root kontrolü → kullanıcı adı al → yorum al → rastgele şifre üret → hesabı oluştur → şifreyi ata → şifreyi süresi dolmuş yap → sonucu ekrana bas**, her adımda hata kontrolüyle.